

Class03f1_MNIST

- **MNIST** (Modified National Institute of Standards and Technology) has a collection of 70,000 grayscale images of handwritten digits (0 to 9) used to train and test machine learning models.

- It is a **famous dataset** often used for demonstrating deep learning. The MNIST dataset comprises **60,000 training examples** and **10,000 test examples** of the handwritten digits 0–9, formatted as **28x28-pixel** monochrome images.



Why is MNIST Famous ?

- **The "Hello World" of Machine Learning:** MNIST serves as the universal baseline benchmarking dataset for introducing beginners to computer vision, image classification, and neural networks.
- **Historical Significance:** Released in **1998** by Yann LeCun ..., it was used to prove the real-world viability of **Convolutional Neural Networks (CNNs)** through LeNet-5, paving the way for modern deep learning.
- **Standardized Benchmarking:** For over a decade, every major computer vision algorithm was tested against MNIST to prove its accuracy and efficiency before moving on to more complex tasks.
- **Simplicity and Accessibility:** only simple computer needed

Class03f1_MNIST

**How to run: Double click
Class03f1_MNIST.ipynb
to open !**

```
# Important libraries for running in jupyter notebook
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
# from keras.layers.convolutional import Conv2D #OLD
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
# from keras.utils import np_utils #OLD
from tensorflow.keras.utils import to_categorical
from tensorflow.keras import backend as K
```

```
""" Below was used by Colab
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Dropout
from keras.layers import Flatten
# from keras.layers.convolutional import Conv2D #OLD
from keras.layers import Conv2D
from keras.layers import MaxPooling2D
# from keras.utils import np_utils #OLD
from keras.utils import to_categorical
from keras import backend as K
"""
```



```
# Sample Code Learning: # one hot encode
```

```
y_train = np.array([4, 0, 3, 2])
```

```
print(y_train)
```

```
print('y_train.shape:', y_train.shape)
```

```
Y_train = np_utils.to_categorical(y_train)
```

```
print(Y_train)
```

```
Y_train.shape
```

```
print('Y_train.shape:', Y_train.shape)
```



```
[4 0 3 2]
```

```
y_train.shape: (4,)
```

```
[[0. 0. 0. 0. 1.] ← 4
```

```
[1. 0. 0. 0. 0.] ← 0
```

```
[0. 0. 0. 1. 0.] ← 3
```

```
[0. 0. 1. 0. 0.] ← 2
```

```
Y_train.shape: (4, 5) ← To be used as output matrix
```

Example from the MNIST database

```
import keras
from keras.datasets import mnist
```

To import the MNIST dataset

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train.shape          # (60000, 28, 28)
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255

print('x_train shape:', x_train.shape)
print('x_test shape:', x_test.shape)
print(x_train.shape[0], 'train samples') #60000 train samples
print(x_test.shape[0], 'test samples')

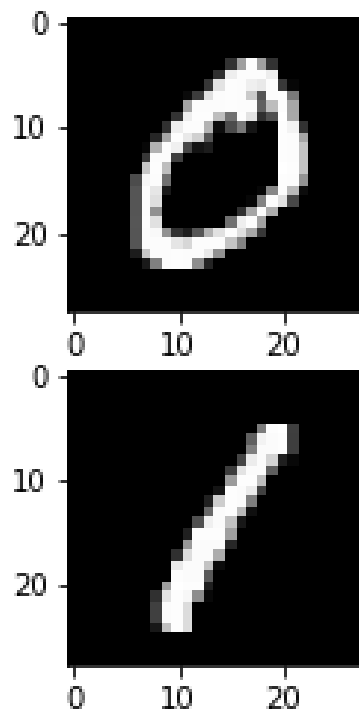
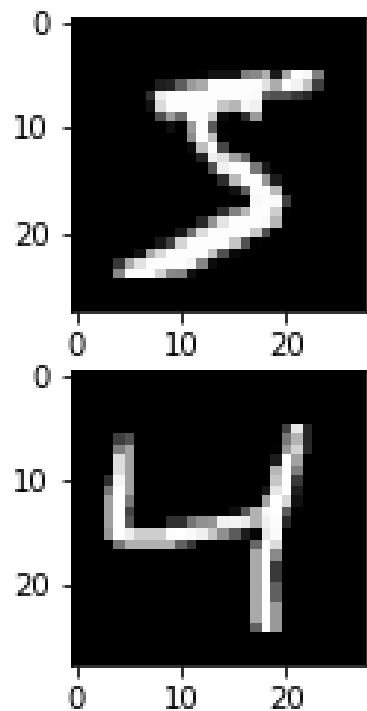
print(x_train[0]) # To show the first image of (28,28) in x_train
```

To readin the MNIST dataset
from publicly available location

```
-----
x_train shape: (60000, 28, 28)
x_test shape: (10000, 28, 28)
60000 train samples
10000 test samples
```

```
plt.subplot(221)
plt.imshow(x_train[0], cmap=plt.get_cmap('gray'))
plt.subplot(222)
plt.imshow(x_train[1], cmap=plt.get_cmap('gray'))
plt.subplot(223)
plt.imshow(x_train[2], cmap=plt.get_cmap('gray'))
plt.subplot(224)
plt.imshow(x_train[3], cmap=plt.get_cmap('gray'))
# show the plot
plt.show()
```

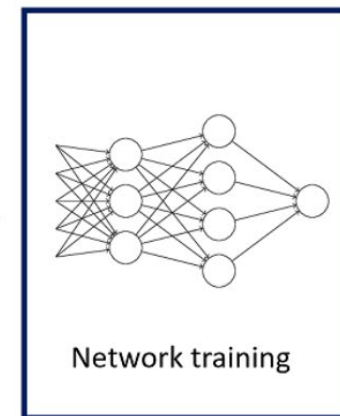
Two methods for digit classification: ANN or CNN



0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
 6 6 6 6 6 6 6 6 6 6 6 6 6 6 6
 7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9



Data & Labels



0
1
2
3
4
5
6
7
8
9



```
# Artificial Neural Network for MNIST
```

```
# fix random seed for reproducibility
```

```
seed = 7
```

```
np.random.seed(seed)
```

```
# reshape to be [samples][pixels][width][height]
```

```
X_train = x_train.reshape(x_train.shape[0], 1, 28, 28).astype('float32')
```

```
X_test = x_test.reshape(x_test.shape[0], 1, 28, 28).astype('float32')
```

```
print('X_train.shape:', X_train.shape)
```

```
print('X_test.shape:', X_test.shape)
```

```
print('original label')
```

```
print(y_train)
```

```
print('y_train.shape:', y_train.shape)
```

```
X_train.shape: (60000, 1, 28, 28) ← a greyscale image
```

```
X_test.shape: (10000, 1, 28, 28)
```

```
original label
```

```
[5 0 4 ... 5 6 8]
```

```
y_train.shape: (60000,)
```

```
# labels are changed a one_hot representation.
```

```
Y_train = to_categorical(y_train)
```

```
print('new label')
```

```
print(Y_train)
```

```
Y_train.shape
```

```
print('Y_train.shape:', Y_train.shape)
```

```
Y_test = to_categorical(y_test)
```

```
num_classes = Y_test.shape[1]
```

← One hot encoding
for the target output

```
new label
```

```
[[0. 0. 0. ... 0. 0. 0.]
```

```
 [1. 0. 0. ... 0. 0. 0.]
```

```
 [0. 0. 0. ... 0. 0. 0.]
```

```
...
```

```
 [0. 0. 0. ... 0. 0. 0.]
```

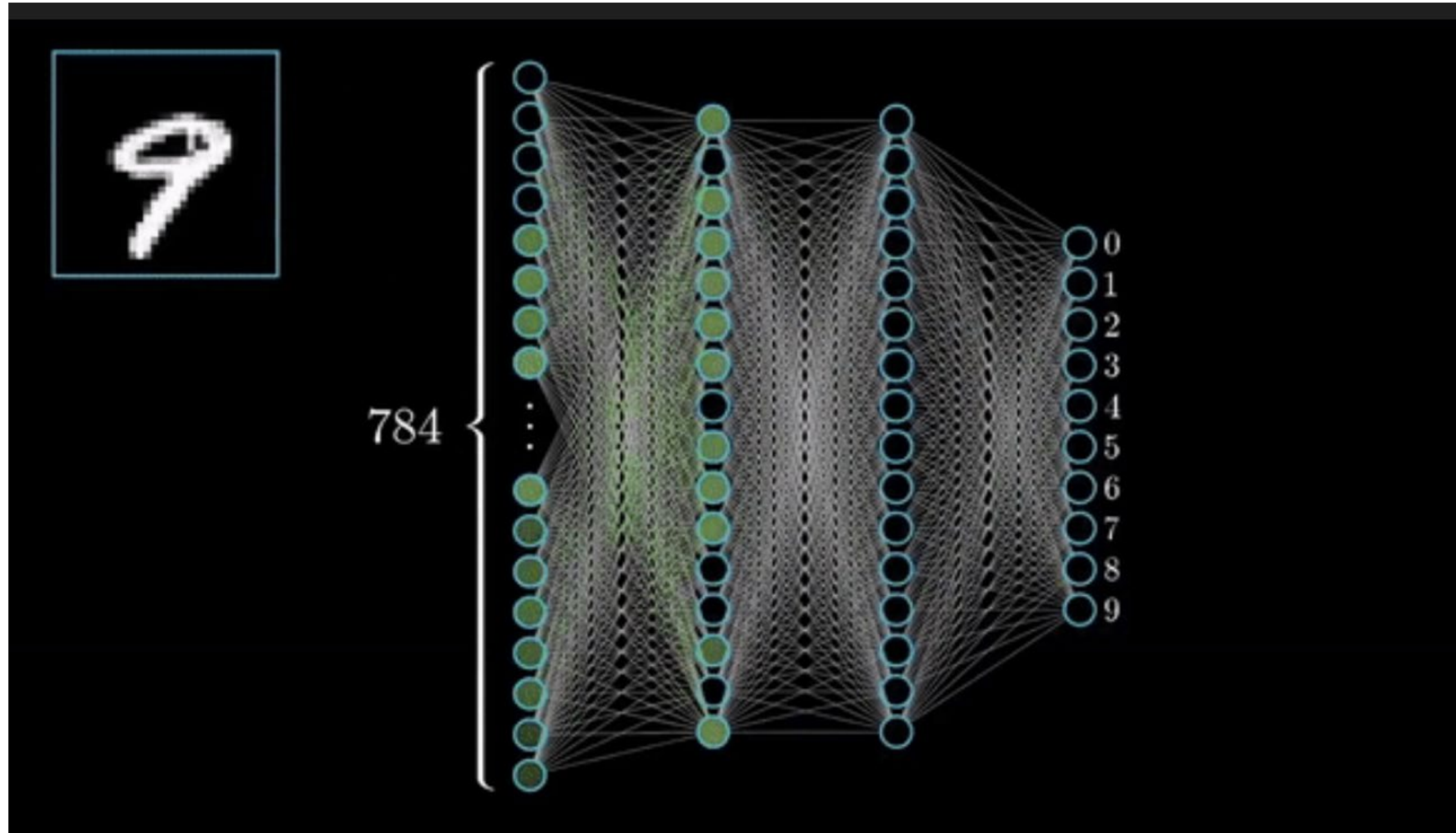
```
 [0. 0. 0. ... 0. 0. 0.]
```

```
 [0. 0. 0. ... 0. 1. 0.]
```

```
Y_train.shape: (60000, 10)
```

← One hot encoding
for the target output

Example 1 (ANN): 784 inputs, 10 outputs





```
# EXAMPLE 1:
# ANN example based on 784 (28x28) inputs, 1D input
#
from keras.datasets import mnist

# load the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# fix random seed for reproducibility
seed = 7
np.random.seed(seed)
#
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
#
# flatten 28*28 images to a 784 vector for each image
print('X_train.shape before reshape' , X_train.shape)
num_pixels = X_train.shape[1] * X_train.shape[2]

X_train = X_train.reshape(X_train.shape[0], num_pixels).astype('float32')
X_test = X_test.reshape(X_test.shape[0], num_pixels).astype('float32')

print('X_train.shape after reshape' , X_train.shape)
```

1-D of 784 values

```
# normalize inputs from 0-255 to 0-1
```

```
X_train = X_train / 255
```

```
X_test = X_test / 255
```

```
# one hot encode outputs
```

```
y_train = to_categorical(y_train)
```

```
y_test = to_categorical(y_test)
```

```
num_classes = y_test.shape[1]
```

```
num_hidden_nodes = 650
```

← One hot encoding
for the target output

← 10 output nodes

Note:

```
X_train.shape before reshape (60000, 28, 28)
```

```
X_train.shape after reshape (60000, 784)
```

← 784 input nodes

← 650 hidden nodes

```
# define baseline model
# Dense : a regular fully connected layer
# The chosen compile model has a loss function, and the purpose is to compute the quantity that
# the model should minimize during training. Below, the 'categorical_crossentropy' is used as
# the loss function. (The MSE is not chosen to be the loss function.) It would compute the crossentropy
# loss between the labels and predictions. There are two or more label classes, usually with labels
# provided in a one_hot representation.
```

Keras framework; 1 hidden layer

```
def baseline_model():
    # create model
    model = Sequential()
    model.add(Dense(num_hidden_nodes, input_dim=num_pixels, kernel_initializer='normal', activation='relu'))
    model.add(Dense(num_classes, kernel_initializer='normal', activation='softmax'))
    # Compile model
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    # Choice of optimizer: adam (adaptive moment estimation), AdaGrad (adaptive learning rate),
    # sgd (Stochastic gradient descent), RMSprop (similar to AdaGrad), Adadelta (adaptive delta) ...
    return model
# build the model
model = baseline_model()
```

650 hidden nodes

784 input pixels

adaptive learning rate

Loss function: 10 output units

10 output units

```
# Fit the model
# verbose : number of steps before showing a result
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=25, batch_size=200, verbose=2)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("Baseline Error: %.2f%%" % (100-scores[1]*100))
#
metrics = history.history
plt.plot(history.epoch, metrics['accuracy'], metrics['val_accuracy'])
plt.legend(['accuracy', 'val_accuracy'])
plt.show()

plt.plot(history.epoch, metrics['loss'], metrics['val_loss'])
plt.legend(['loss', 'val_loss'])
plt.show()
```

Note: epoch, batch_size

To store the training info

**To obtain the training
info and use for
plotting**

Epoch 1/25

300/300 - 3s - loss: 0.2968 - accuracy: 0.9158 - val_loss: 0.1393 - val_accuracy: 0.9585 - 3s/epoch - 11ms/step

Epoch 2/25

300/300 - 1s - loss: 0.1203 - accuracy: 0.9659 - val_loss: 0.1012 - val_accuracy: 0.9701 - 722ms/epoch - 2ms/step

Epoch 3/25

300/300 - 1s - loss: 0.0780 - accuracy: 0.9775 - val_loss: 0.0779 - val_accuracy: 0.9753 - 783ms/epoch - 3ms/step

Epoch 4/25

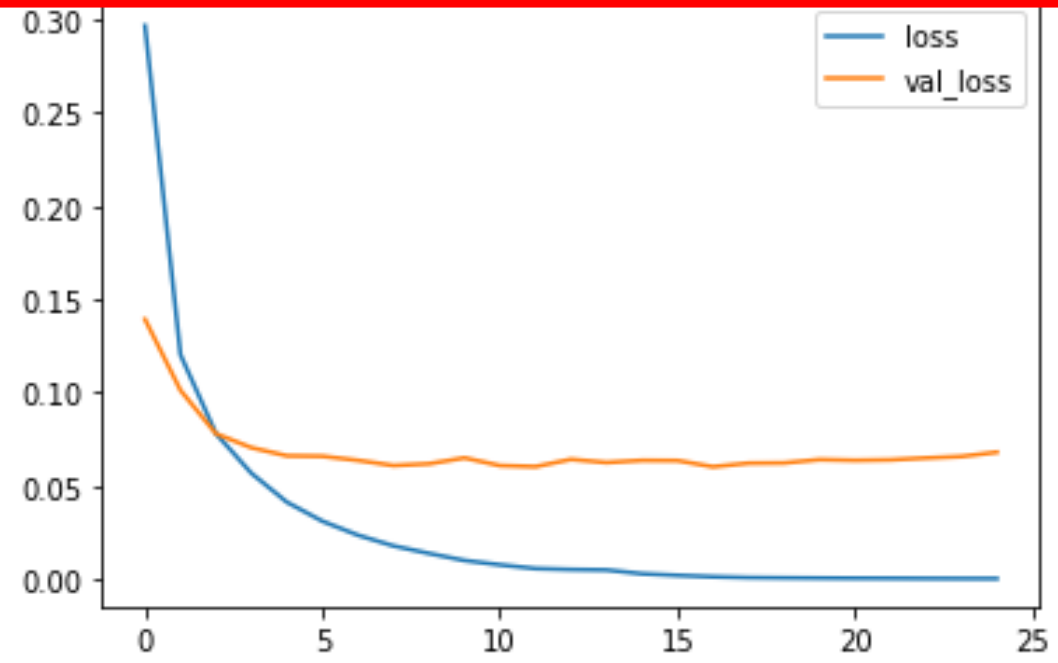
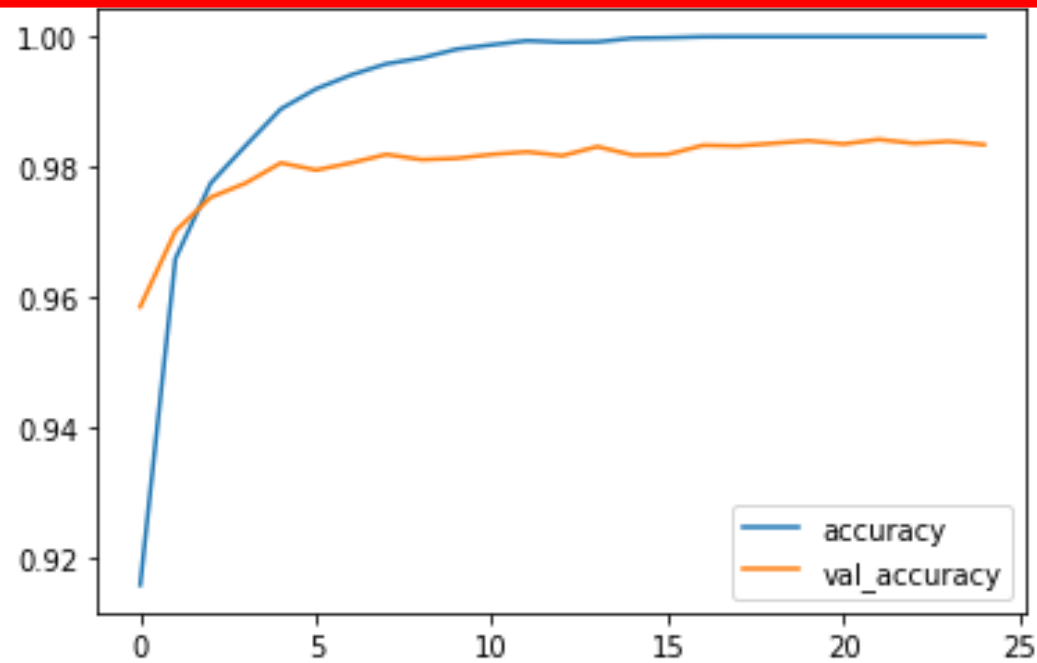
300/300 - 1s - loss: 0.0567 - accuracy: 0.9833 - val_loss: 0.0704 - val_accuracy: 0.9775 - 745ms/epoch - 2ms/step

Epoch 5/25

Epoch 25/25

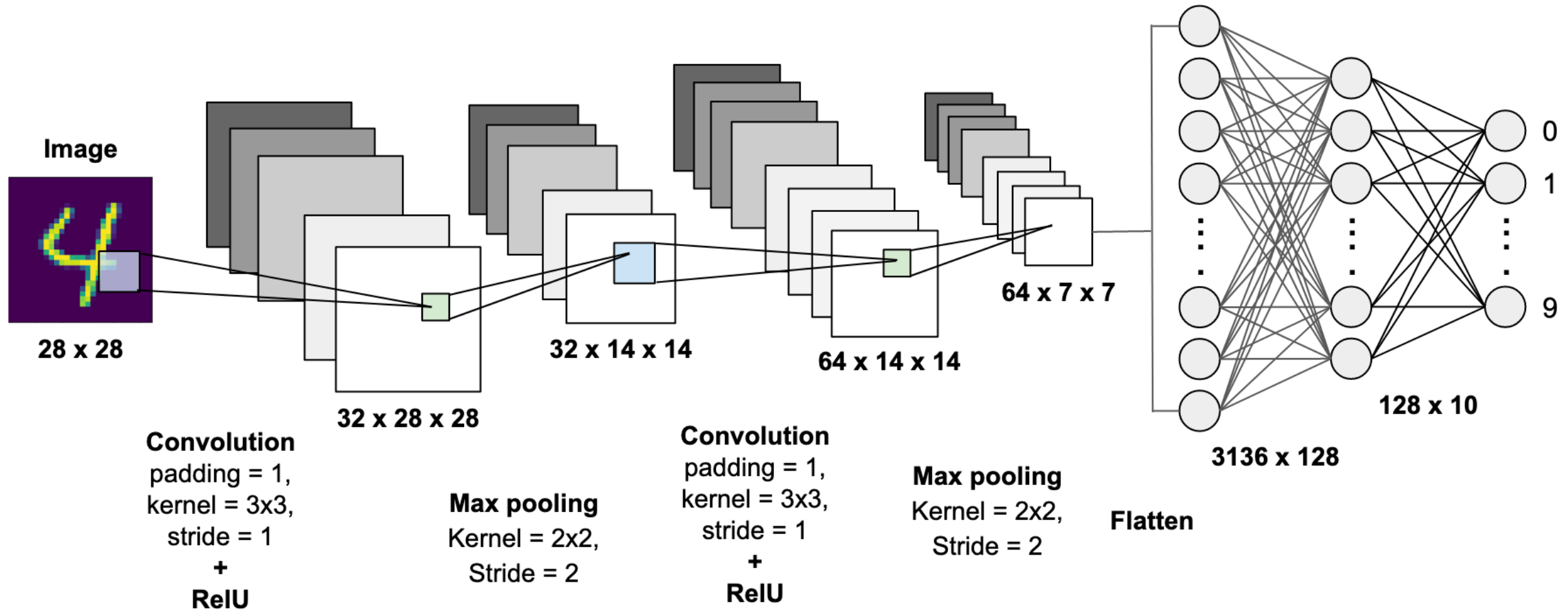
300/300 - 1s - loss: 2.4086e-04 - accuracy: 1.0000 - val_loss: 0.0680 - val_accuracy: 0.9834 - 693ms/epoch - 2ms/s

Baseline Error: 1.66%



Error is 1.66% of 10000 test images.

Example 2 (CNN): 2D inputs (28x28x1) , 10 outputs



```
# EXAMPLE 2:  
# The example based on a simple CNN structure, 2D input  
# jupyter code:  
from tensorflow.keras.layers import Conv2D  
from tensorflow.keras.layers import MaxPooling2D
```

```
seed = 7  
np.random.seed(seed)  
# Load data
```

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()  
# reshape to be [samples][pixels][width][height]  
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1).astype('float32')  
X_test = X_test.reshape(X_test.shape[0], 28, 28, 1).astype('float32')
```

```
# normalize inputs from 0-255 to 0-1  
X_train = X_train / 255  
X_test = X_test / 255
```

```
# one hot encode outputs  
y_train = to_categorical(y_train)  
y_test = to_categorical(y_test)  
num_classes = y_test.shape[1]
```

```
# define baseline model
```

```
def baseline_model():
```

```
    # create model
```

```
    model = Sequential()
```

```
    model.add(Conv2D(32, (5, 5), input_shape=(28, 28,1), activation='relu'))
```

```
    model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
    model.add(Dropout(0.2))
```

```
    model.add(Flatten())
```

```
    model.add(Dense(128, activation='relu'))
```

```
    model.add(Dense(num_classes, activation='softmax'))
```

```
    # Compile model
```

```
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
    return model
```

```
#
```

```
# build the model
```

```
model = baseline_model()
```

```
# Fit the model
```

```
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200, verbose=2)
```

2D inputs (28x28x1)



```
Epoch 1/10
```

```
300/300 - 11s - loss: 0.2476 - accuracy: 0.9276 - val_loss: 0.0869 - val_accuracy: 0.9759 - 11s/epoch - 37ms/step
```

```
Epoch 2/10
```

```
300/300 - 1s - loss: 0.0773 - accuracy: 0.9767 - val_loss: 0.0563 - val_accuracy: 0.9829 - 1s/epoch - 4ms/step
```

```
Epoch 3/10
```

```
300/300 - 1s - loss: 0.0534 - accuracy: 0.9843 - val_loss: 0.0429 - val_accuracy: 0.9855 - 1s/epoch - 4ms/step
```

```
# Final evaluation of the model

scores = model.evaluate(X_test, y_test, verbose=0)
print("CNN Error: %.2f%%" % (100-scores[1]*100))
#
```

```
Epoch 10/10
300/300 - 1s - loss: 0.0152 - accuracy: 0.9952 - val_loss: 0.0284 - val_accuracy: 0.9897 - 1s/epoch - 4ms/step
CNN Error: 1.03%
```

Error is 1.03% of 10000 test images.



EXAMPLE 3: Example 3 (CNN): 2D inputs (28x28x1) , 10 outputs

The example based on another simple CNN structure, 2D input

Colab codes

```
from keras.layers import Conv2D
from keras.layers import MaxPooling2D
```

```
seed = 7
np.random.seed(seed)
# load data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# reshape to be [samples][pixels][width][height]
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1).astype('float32')
X_test = X_test.reshape(X_test.shape[0], 28, 28, 1).astype('float32')
# normalize inputs from 0-255 to 0-1
X_train = X_train / 255
X_test = X_test / 255
```

```
# one hot encode outputs
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
```

Jupyter notebook codes

```
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
```

```
def baseline_model():  
    # create model  
    model = Sequential()  
    model.add(Conv2D(64, kernel_size=(3, 3), input_shape=(28, 28, 1), activation='relu'))  
    model.add(MaxPooling2D(pool_size=(2, 2)))  
    model.add(Conv2D(32, kernel_size=(3, 3), activation='relu'))  
    model.add(MaxPooling2D(pool_size=(2, 2)))  
    model.add(Dropout(0.25))  
    model.add(Flatten())  
    model.add(Dense(128, activation='relu'))  
    model.add(Dropout(0.5))  
    model.add(Dense(num_classes, activation='softmax'))  
    # Compile model  
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])  
    return model
```

```
model = baseline_model()
# Fit the model
model.fit(x=X_train, y=y_train, validation_data=(X_test, y_test), epochs=10, batch_size=200, verbose=2)
# Final evaluation of the model
scores = model.evaluate(X_test, y_test, verbose=0)
print("CNN Error: %.2f%%" % (100-scores[1]*100))
```

Epoch 1/10

300/300 - 4s - loss: 0.4312 - accuracy: 0.8637 - val_loss: 0.0790 - val_accuracy: 0.9745 - 4s/epoch - 13ms/step

Epoch 2/10

300/300 - 2s - loss: 0.1349 - accuracy: 0.9595 - val_loss: 0.0513 - val_accuracy: 0.9830 - 2s/epoch - 6ms/step

Epoch 3/10

300/300 - 2s - loss: 0.1018 - accuracy: 0.9691 - val_loss: 0.0396 - val_accuracy: 0.9869 - 2s/epoch - 6ms/step

Epoch 4/10

300/300 - 2s - loss: 0.0830 - accuracy: 0.9754 - val_loss: 0.0338 - val_accuracy: 0.9888 - 2s/epoch - 6ms/step

Epoch 5/10

300/300 - 2s - loss: 0.0702 - accuracy: 0.9787 - val_loss: 0.0314 - val_accuracy: 0.9893 - 2s/epoch - 7ms/step

Epoch 6/10

300/300 - 2s - loss: 0.0632 - accuracy: 0.9809 - val_loss: 0.0303 - val_accuracy: 0.9900 - 2s/epoch - 8ms/step

Epoch 7/10

300/300 - 2s - loss: 0.0585 - accuracy: 0.9818 - val_loss: 0.0260 - val_accuracy: 0.9913 - 2s/epoch - 7ms/step

Epoch 8/10

300/300 - 2s - loss: 0.0540 - accuracy: 0.9833 - val_loss: 0.0245 - val_accuracy: 0.9917 - 2s/epoch - 6ms/step

Epoch 9/10

300/300 - 2s - loss: 0.0487 - accuracy: 0.9845 - val_loss: 0.0241 - val_accuracy: 0.9913 - 2s/epoch - 6ms/step

Epoch 10/10

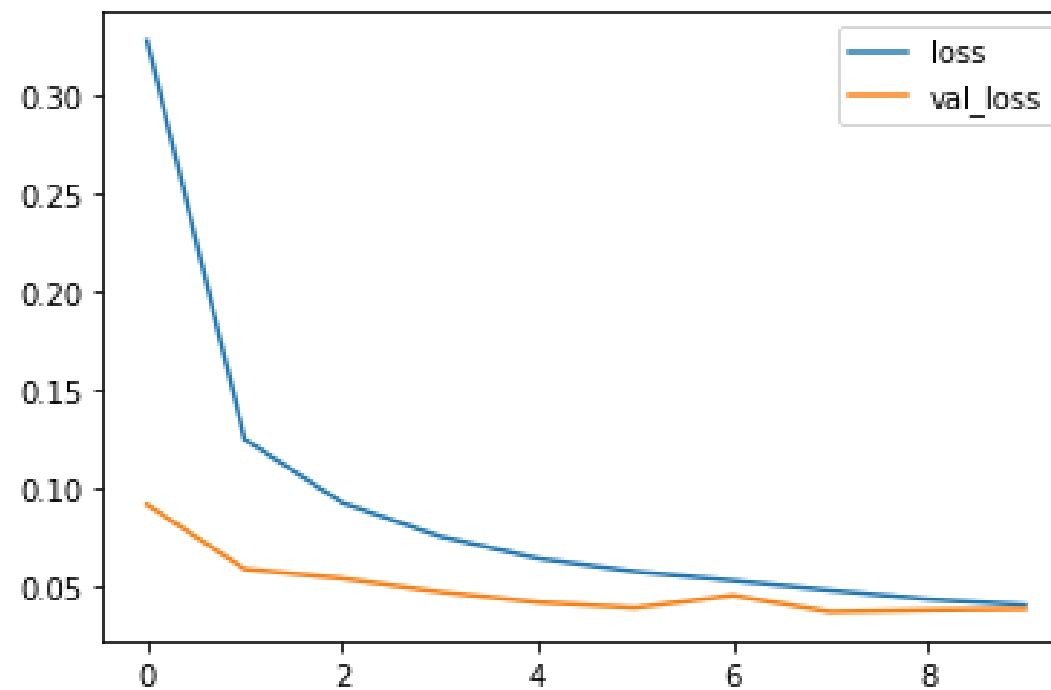
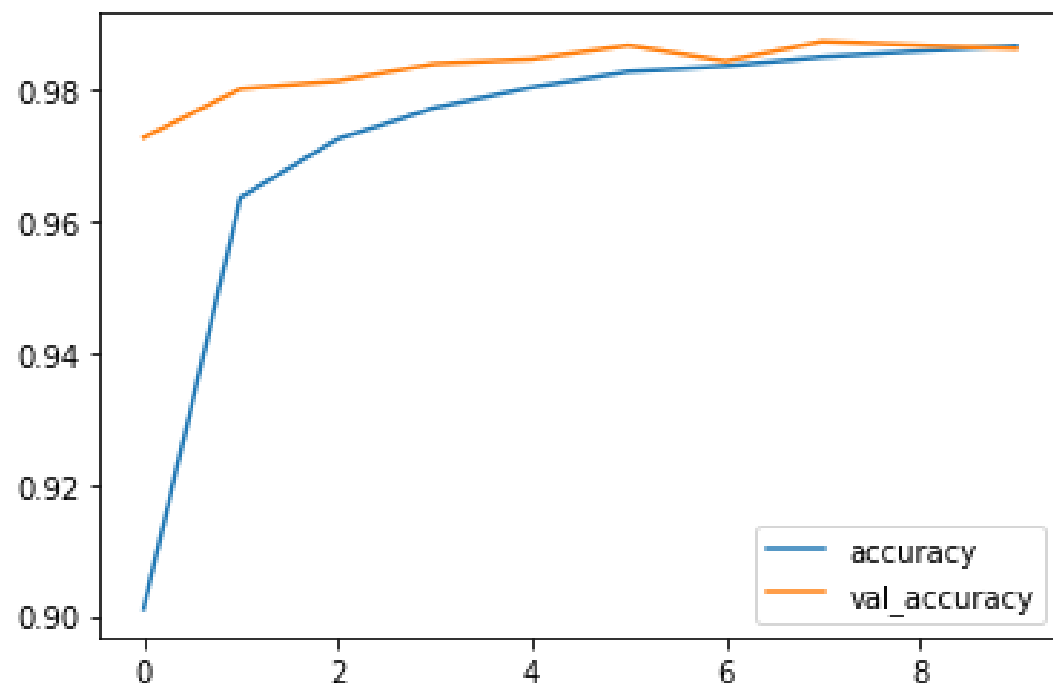
300/300 - 2s - loss: 0.0469 - accuracy: 0.9855 - val_loss: 0.0239 - val_accuracy: 0.9921 - 2s/epoch - 6ms/step

CNN Error: 0.79%

Error is 0.79% of 10000 test images.

Add the code to generate the following graphs

What do you notice in the graphs ?



<https://medium.com/data-science/mnist-handwritten-digits-classification-using-a-convolutional-neural-network-cnn-af5fafbc35e9>